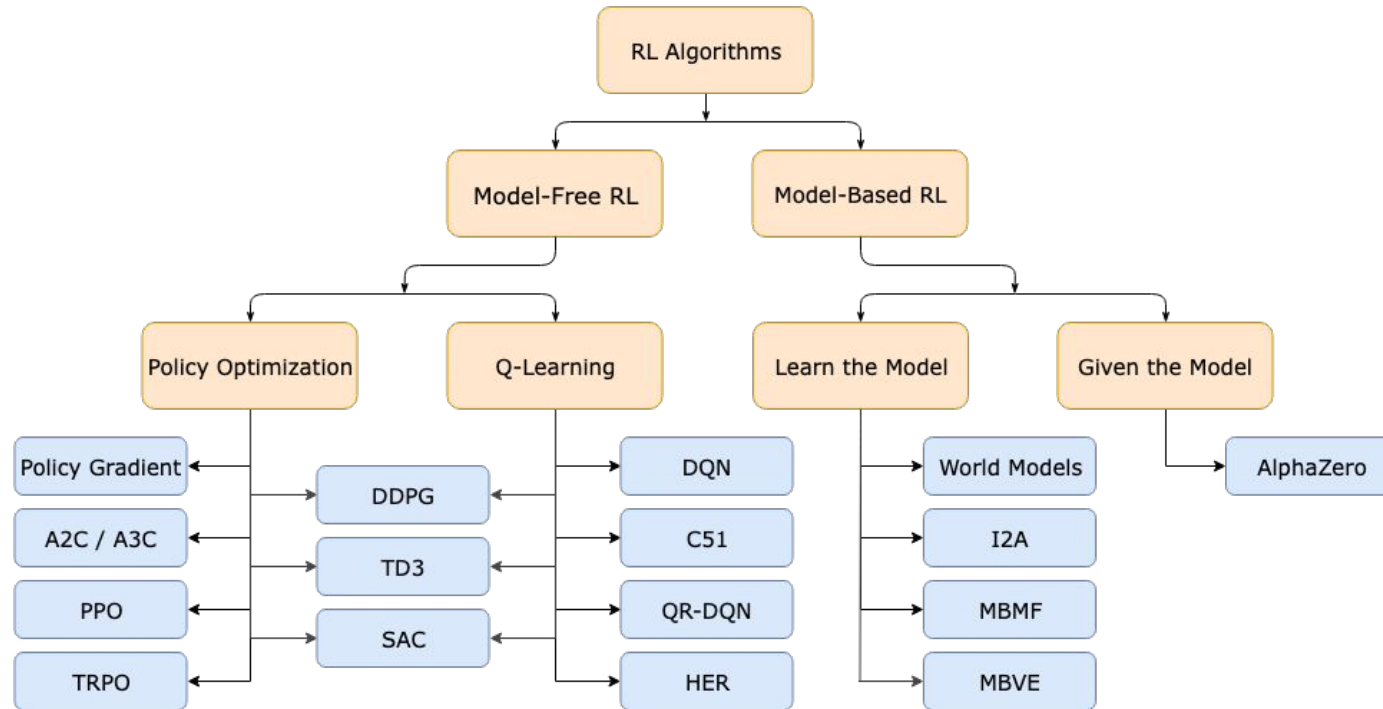


Policy Gradient Methods

Types of RL Algorithms



Types of RL Algorithms

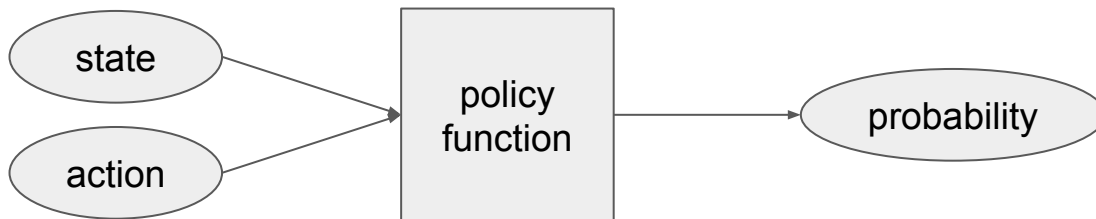


- So far, we have covered so-called “Action-value” methods (e.g. Q-learning)
- They learned the values of actions and then selected actions based on their estimated action value
- Is there any other way?
- Yes, policy gradient methods!
- Learn a parameterized policy that can select actions without consulting a value function

Parameterized Policy

- What is a policy?
- A policy is a function to map a state-action pair to a probability to choose the action at the state
- Again, can we approximate the policy function?

$$\pi(a|s, \theta) = \Pr\{A_t=a \mid S_t=s, \theta_t=\theta\}$$



Policy Gradient Methods: General Schema



1. Define a performance function $J(\theta)$ that indicates the performance of the policy parameterized by θ
2. Solve a maximization problem of the performance function by taking Gradient ascent steps

$$\theta_{t+1} = \theta_t + \alpha \widehat{\nabla J(\theta_t)}$$

How do you solve a maximization problem?



- A simple mathematical solution

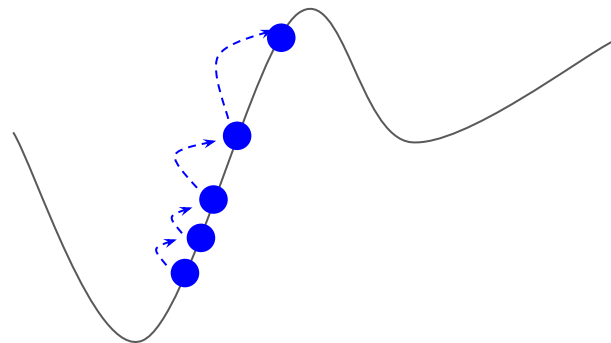
$$J(w) = -w^2$$

- Differentiate J with respect to w
- It is not always possible with a complex objective function
- Also, it tends to be computationally expensive

How do you solve a maximization problem?

- Gradient Ascent (GA)
 - Starting from an initial point $\mathbf{w}_0$
 - Always find another point $\mathbf{w}_{t+1}$ that the corresponding function value increases, compared to the one at the previous point $\mathbf{w}_t$

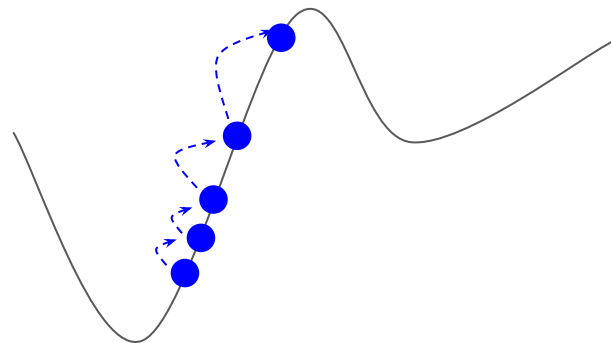
$$J(w_{t+1}) \geq J(w_t)$$



How do you solve a minimization problem?

- The gradient $\nabla J(\mathbf{w})$ of a scalar-valued differentiable function J at a point $\mathbf{w}$

$$\nabla J(\mathbf{w}) = \begin{bmatrix} \frac{\partial J}{\partial w_1}(\mathbf{w}) \\ \vdots \\ \frac{\partial J}{\partial w_n}(\mathbf{w}) \end{bmatrix}$$



Milestone Questions: Gradient



Compute the gradient of the following function

$$J(w_1, w_2) = -w_1^2 + w_1 w_2$$

Agenda



1. How to parameterize a policy function
 - Any parameterization should work as long as the function π is differentiable with respect to its parameters
 - Two examples of the parameterization
 - Discrete action space: Softmax in action preferences
 - Continuous action space: Gaussian distribution
2. How to define a policy performance function J

Softmax in action preferences



- Assume: the action space is discrete and not too large
- Define parameterized numerical preferences (a larger value indicates that the state action pair is more preferred)

$$h(s, a, \boldsymbol{\theta})$$

- We compute the corresponding probabilities based on the preferences, using the softmax function

$$\pi(a|s, \boldsymbol{\theta}) \doteq \frac{e^{h(s,a,\boldsymbol{\theta})}}{\sum_b e^{h(s,b,\boldsymbol{\theta})}}$$

Milestone Question: An example



- Two actions: a1 and a2
- $h(s, a1, \theta) = 10$
- $h(s, a2, \theta) = 2$
- Compute the probability of choosing a1

$$\pi(a|s, \theta) \doteq \frac{e^{h(s,a,\theta)}}{\sum_b e^{h(s,b,\theta)}}$$

Milestone Question: An example



- Two actions: a1 and a2
- $h(s, a1, \theta) = 10$
- $h(s, a2, \theta) = 2$
- Compute the probability of choosing a1
- $\exp(10) / [\exp(10) + \exp(2)] = 0.9996646499$

$$\pi(a|s, \theta) \doteq \frac{e^{h(s,a,\theta)}}{\sum_b e^{h(s,b,\theta)}}$$

Softmax in action preferences



- The numerical preferences can be parameterized arbitrarily
 - Neural network
 - A linear combination of features
 - ...
- Here, for simplicity, we look at the linear function case

$$h(s, a, \boldsymbol{\theta}) = \boldsymbol{\theta}^\top \mathbf{x}(s, a)$$

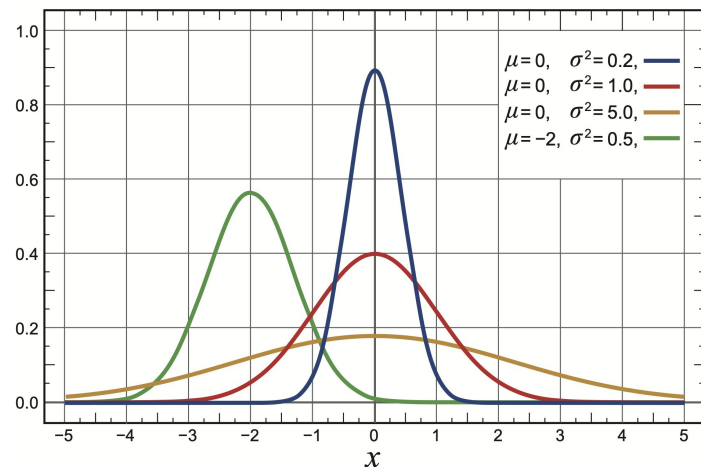
$\mathbf{x}(s, a)$ is a feature representation of a state-action pair (s, a)

Gaussian distribution

- Instead of computing learned probabilities for each of the many actions, we can learn statistics of the probability distribution
- For example, when we can assume that there is a set of similar actions that perform well, we can use a Gaussian distribution
- The probability density function

$$p(x) \doteq \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

- Here, only factors to examine are mean and standard deviation



Note: the y-axis shows the “density” not probability. The area under a curve sums to 1.0.

- Therefore, the parametric policy function is

$$\pi(a|s, \boldsymbol{\theta}) \doteq \frac{1}{\sigma(s, \boldsymbol{\theta})\sqrt{2\pi}} \exp\left(-\frac{(a - \mu(s, \boldsymbol{\theta}))^2}{2\sigma(s, \boldsymbol{\theta})^2}\right)$$

- In a parameter vector, one part to be used for the approximation of the mean and one part for the approximation of the standard deviation

$$\boldsymbol{\theta} = [\boldsymbol{\theta}_\mu, \boldsymbol{\theta}_\sigma]^\top$$

- e.g., The mean can be approximated as a linear function; The standard deviation must always be positive and is better approximated as the exponential of a linear function

$$\mu(s, \boldsymbol{\theta}) \doteq \boldsymbol{\theta}_\mu^\top \mathbf{x}_\mu(s) \quad \text{and} \quad \sigma(s, \boldsymbol{\theta}) \doteq \exp\left(\boldsymbol{\theta}_\sigma^\top \mathbf{x}_\sigma(s)\right)$$

Milestone Question: An example



- $\theta_\mu = [1, 1]$
- $\theta_\sigma = [0.5, 1]$
- $\mathbf{x}_\mu(s_1) = [1, 2]$
- $\mathbf{x}_\sigma(s_1) = [0, 1]$
- Compute the probability of choosing $a = 1.0$

$$\pi(a|s, \boldsymbol{\theta}) \doteq \frac{1}{\sigma(s, \boldsymbol{\theta})\sqrt{2\pi}} \exp\left(-\frac{(a - \mu(s, \boldsymbol{\theta}))^2}{2\sigma(s, \boldsymbol{\theta})^2}\right)$$

$$\mu(s, \boldsymbol{\theta}) \doteq \boldsymbol{\theta}_\mu^\top \mathbf{x}_\mu(s)$$

$$\sigma(s, \boldsymbol{\theta}) \doteq \exp\left(\boldsymbol{\theta}_\sigma^\top \mathbf{x}_\sigma(s)\right)$$

Agenda

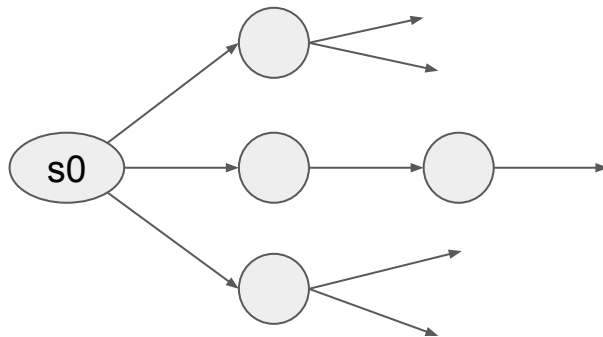


1. How to parameterize a policy function
 - Any parameterization should work as long as the function π is differentiable with respect to its parameters
 - Two examples of the parameterization
 - Discrete action space: Softmax in action preferences
 - Continuous action space: Gaussian distribution
2. How to define a policy performance function J

Policy Performance Function J

- In the episodic case, we can define the performance as

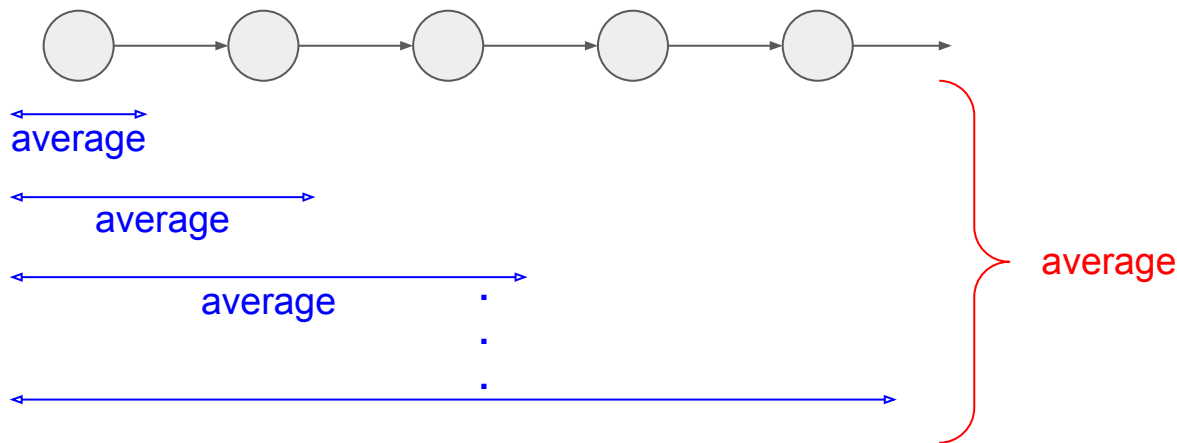
$$J(\boldsymbol{\theta}) \doteq v_{\pi_{\boldsymbol{\theta}}}(s_0)$$



Policy Performance Function J

- In the continuing case, we can think about the concept of average reward of each time step t

$$J(\theta) \doteq r(\pi) \doteq \lim_{h \rightarrow \infty} \frac{1}{h} \sum_{t=1}^h \mathbb{E}[R_t \mid S_0, A_{0:t-1} \sim \pi]$$



Policy Performance Function J



- In an ergodic MDP,
 - where the steady-state distribution $\mu(s) \doteq \lim_{t \rightarrow \infty} \Pr\{S_t = s | A_{0:t} \sim \pi\}$ exists and is independent of S_0
 - i.e., the starting state and any early decision made by the agent can have only a temporary effect; in the long run the expectation of being in a state depends only on the policy and the MDP transition probabilities

$$\begin{aligned} J(\boldsymbol{\theta}) &\doteq r(\pi) \doteq \lim_{h \rightarrow \infty} \frac{1}{h} \sum_{t=1}^h \mathbb{E}[R_t \mid S_0, A_{0:t-1} \sim \pi] \\ &= \lim_{t \rightarrow \infty} \mathbb{E}[R_t \mid S_0, A_{0:t-1} \sim \pi] \\ &= \sum_s \mu(s) \sum_a \pi(a|s) \sum_{s', r} p(s', r | s, a) r \end{aligned}$$

The Policy Gradient Theorem



- Provides an analytic expression for the gradient of performance J with respect to the policy parameter θ
- For the episodic case,

$$\nabla J(\theta) \propto \sum_s \mu(s) \sum_a q_\pi(s, a) \nabla \pi(a|s, \theta)$$

“proportional to”

$$= \mathbb{E}_\pi \left[\sum_a q_\pi(S_t, a) \nabla \pi(a|S_t, \theta) \right]$$

- When sampling episodes, the distribution of state visits should be represented in the simple average based on a current policy

- Similarly, we can replace the sum over all actions a with a sampled A_t in an experience

$$\begin{aligned}\nabla J(\boldsymbol{\theta}) &\propto \mathbb{E}_{\pi} \left[\sum_a \pi(a|S_t, \boldsymbol{\theta}) q_{\pi}(S_t, a) \frac{\nabla \pi(a|S_t, \boldsymbol{\theta})}{\pi(a|S_t, \boldsymbol{\theta})} \right] \\ &= \mathbb{E}_{\pi} \left[q_{\pi}(S_t, A_t) \frac{\nabla \pi(A_t|S_t, \boldsymbol{\theta})}{\pi(A_t|S_t, \boldsymbol{\theta})} \right] && \text{(replacing } a \text{ by the sample } A_t \sim \pi) \\ &= \mathbb{E}_{\pi} \left[G_t \frac{\nabla \pi(A_t|S_t, \boldsymbol{\theta})}{\pi(A_t|S_t, \boldsymbol{\theta})} \right], && \text{(because } \mathbb{E}_{\pi}[G_t|S_t, A_t] = q_{\pi}(S_t, A_t))\end{aligned}$$

REINFORCE: Monte Carlo Policy Gradient



- Based on the previous derivation of $\nabla J(\theta)$,

$$\theta_{t+1} \doteq \theta_t + \alpha G_t \frac{\nabla \pi(A_t|S_t, \theta_t)}{\pi(A_t|S_t, \theta_t)}$$

- The update increases the parameter vector in this direction
 - **proportional to the return** (it causes the parameter to move most in the directions that favor actions that yield the highest return)
 - **inversely proportional to the action probability** (otherwise actions that are selected frequently are at an advantage [the updates will be more often in their direction] and might win out even if they do not yield the highest return)

REINFORCE: Monte-Carlo Policy-Gradient Control (episodic) for π_*

Input: a differentiable policy parameterization $\pi(a|s, \theta)$

Algorithm parameter: step size $\alpha > 0$

Initialize policy parameter $\theta \in \mathbb{R}^{d'}$ (e.g., to $\mathbf{0}$)

Loop forever (for each episode):

Generate an episode $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$, following $\pi(\cdot|\cdot, \theta)$

Loop for each step of the episode $t = 0, 1, \dots, T - 1$:

$$G \leftarrow \sum_{k=t+1}^T \gamma^{k-t-1} R_k \quad (G_t)$$

$$\theta \leftarrow \theta + \alpha \gamma^t G \nabla \ln \pi(A_t|S_t, \theta)$$

Equivalent to $\frac{\nabla \pi(A_t|S_t, \theta_t)}{\pi(A_t|S_t, \theta_t)}$ (It is called an *eligibility vector*. $\nabla \ln x = \frac{\nabla x}{x}$)

Advantages of Policy Approximation



1. The policy may be a simpler function to approximate
 - Action-value methods may overcomplicate the approximation by estimating individual Q values
2. The softmax enable the selection of actions with arbitrary probabilities (next slide!)
 - An optimal policy could be stochastic
 - Action-value methods cannot produce it
 - E.g. Bluffing in poker; Rock-Paper-Scissors
3. You can inject prior knowledge about the desired form of the policy
 - Initialization and the function form can provide a bias (in a good way if designed well)
4. Stronger convergence guarantees are available
 - A series of intermediate policies show the continuity

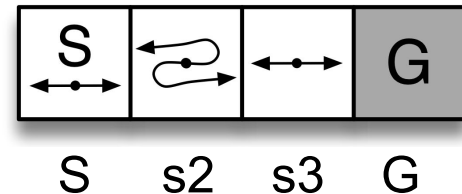
Example: Short corridor with switched actions

- The reward is -1 per step
- There are only two actions, right and left
 - **S & s3**: Usual deterministic consequences (left causes no movement in s1)
 - **s2**: Actions are reversed (right moves to the left and left moves to the right)
- Feature representation of states is not well designed and all the states appear identical; for all states s ,

$$\mathbf{x}(s, \text{right}) = [1, 0]^T$$

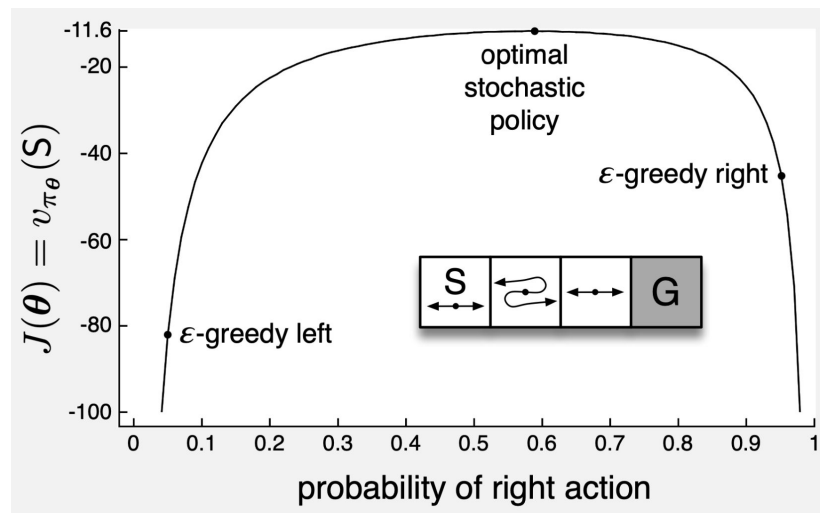
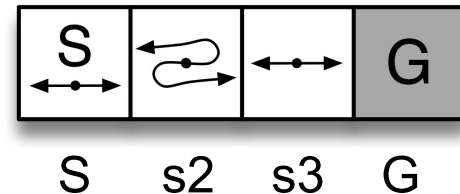
$$\mathbf{x}(s, \text{left}) = [0, 1]^T$$

- Milestone question: What is a good (optimal) policy?

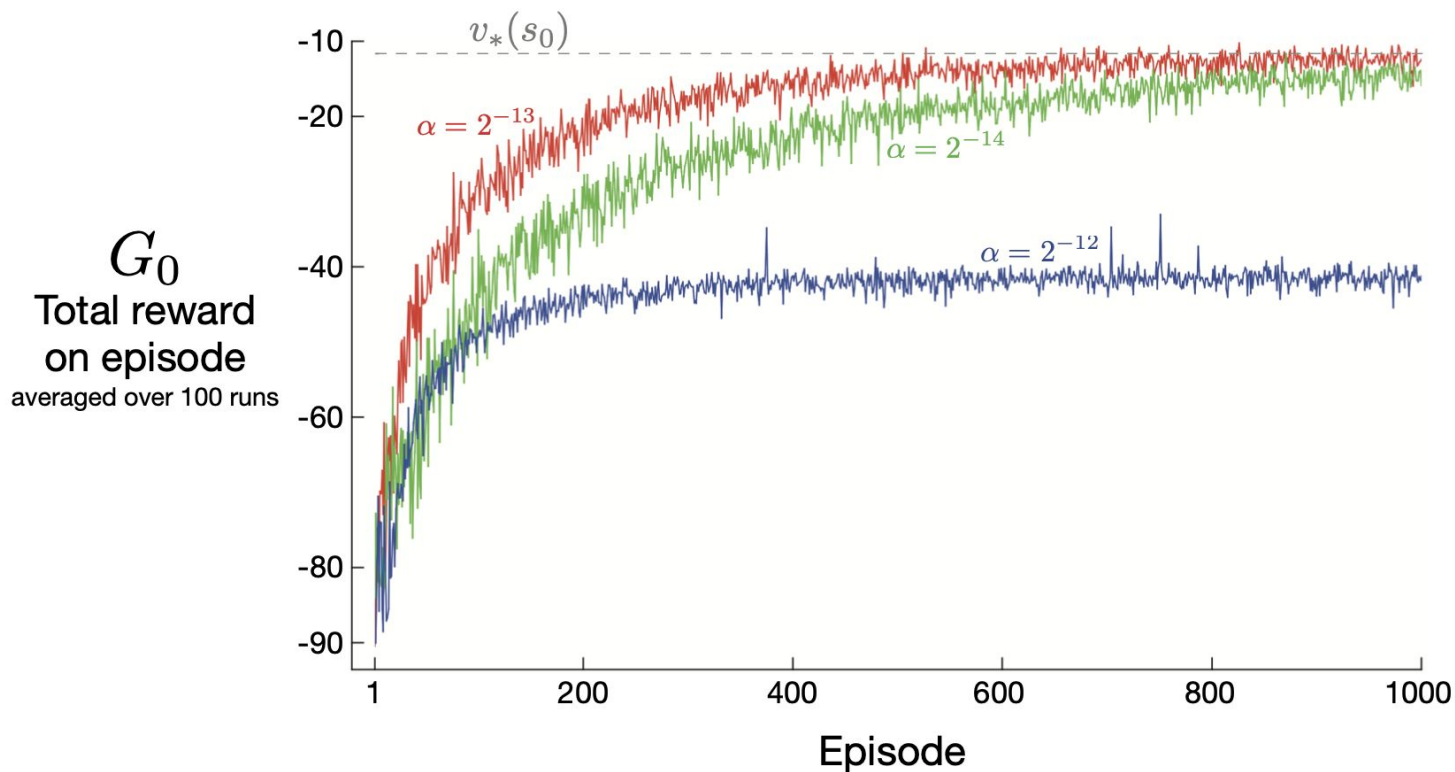


Example: Short corridor with switched actions

- With a policy gradient method, we should be able to find the stochastic policy: $P(\text{RIGHT}) = 0.59$
- An action-value method with "epsilon-greedy action selection is forced to choose between just two policies
 - choosing right with high probability $1 - \epsilon/2$ or
 - choosing left with the same high probability
- The plot shows the values based on $\epsilon = 0.1$



REINFORCE on the Short Corridor



Notes on REINFORCE



- (By construction,) the expected update over an episode is in the same direction as the performance gradient
- This assures
 - an improvement in expected performance for sufficiently small step size (α)
 - convergence to a local optimum under standard stochastic approximation conditions for decreasing step size
- However, as REINFORCE may be of high variance (due to the MC concept) and thus produce slow learning

Advantages of Policy Approximation



1. The policy may be a simpler function to approximate
 - Action-value methods may overcomplicate the approximation by estimating individual Q values
2. The softmax enable the selection of actions with arbitrary probabilities
 - An optimal policy could be stochastic
 - Action-value methods cannot produce it
 - E.g. Bluffing in poker; Rock-Paper-Scissors
3. You can inject prior knowledge about the desired form of the policy
 - Initialization and the function form can provide a bias (in a good way if designed well)
4. Stronger convergence guarantees are available
 - A series of intermediate policies show the continuity

REINFORCE: Monte-Carlo Policy-Gradient Control (episodic) for π_*

Input: a differentiable policy parameterization $\pi(a|s, \theta)$

Algorithm parameter: step size $\alpha > 0$

Initialize policy parameter $\theta \in \mathbb{R}^{d'}$ (e.g., to $\mathbf{0}$)

Loop forever (for each episode):

Generate an episode $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$, following $\pi(\cdot|\cdot, \theta)$

Loop for each step of the episode $t = 0, 1, \dots, T - 1$:

$$G \leftarrow \sum_{k=t+1}^T \gamma^{k-t-1} R_k \quad (G_t)$$

$$\theta \leftarrow \theta + \alpha \gamma^t G \nabla \ln \pi(A_t | S_t, \theta)$$

Equivalent to $\frac{\nabla \pi(A_t | S_t, \theta_t)}{\pi(A_t | S_t, \theta_t)}$ (It is called an *eligibility vector*. $\nabla \ln x = \frac{\nabla x}{x}$)

- We subtract a baseline function $b(s)$ from the policy gradient
- Intuition: A smaller value function makes the gradient smaller (and more stable updates)

$$\nabla J(\boldsymbol{\theta}) \propto \sum_s \mu(s) \sum_a \left(q_\pi(s, a) - b(s) \right) \nabla \pi(a|s, \boldsymbol{\theta})$$

- The baseline can be any function, even a random variable, as long as it does not vary with action a . Why?

$$\sum_a b(s) \nabla \pi(a|s, \boldsymbol{\theta}) = b(s) \nabla \sum_a \pi(a|s, \boldsymbol{\theta}) = b(s) \nabla 1 = 0$$

- One natural choice for the baseline is an estimate of the state value $\hat{v}(S_t, \mathbf{w})$

REINFORCE with Baseline (episodic), for estimating $\pi_{\theta} \approx \pi_*$

Input: a differentiable policy parameterization $\pi(a|s, \theta)$

Input: a differentiable state-value function parameterization $\hat{v}(s, \mathbf{w})$

Algorithm parameters: step sizes $\alpha^{\theta} > 0$, $\alpha^{\mathbf{w}} > 0$

Initialize policy parameter $\theta \in \mathbb{R}^{d'}$ and state-value weights $\mathbf{w} \in \mathbb{R}^d$ (e.g., to $\mathbf{0}$)

Loop forever (for each episode):

Generate an episode $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$, following $\pi(\cdot|\cdot, \theta)$

Loop for each step of the episode $t = 0, 1, \dots, T - 1$:

$$G \leftarrow \sum_{k=t+1}^T \gamma^{k-t-1} R_k \quad (G_t)$$

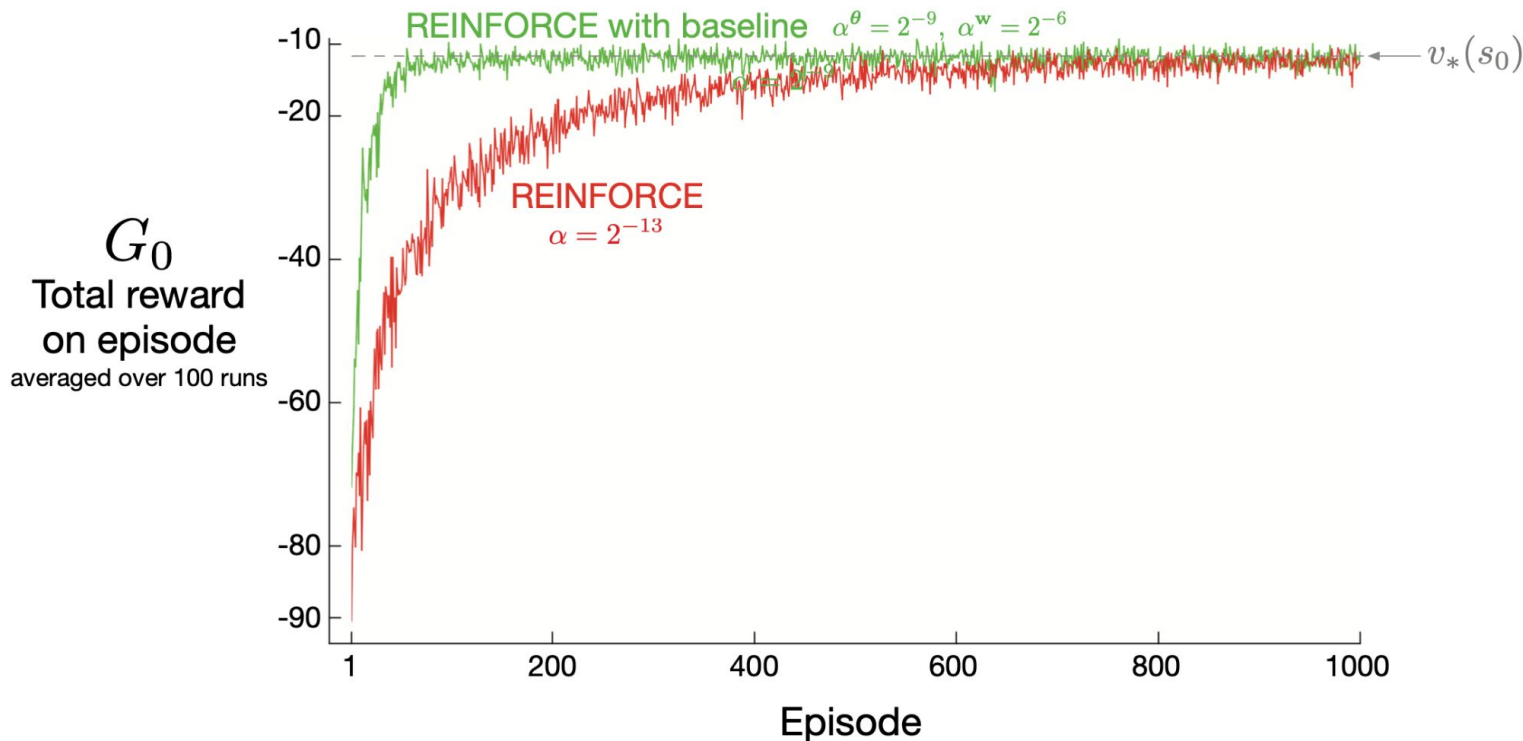
$$\delta \leftarrow G - \hat{v}(S_t, \mathbf{w})$$

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha^{\mathbf{w}} \delta \nabla \hat{v}(S_t, \mathbf{w})$$

$$\theta \leftarrow \theta + \alpha^{\theta} \gamma^t \delta \nabla \ln \pi(A_t|S_t, \theta)$$

Remember the MC-based approximation method

REINFORCE on the Short Corridor



See Part 1 for the problem description.

- Recap: This is the update equation in REINFORCE (MC PG) with Baseline

$$\boldsymbol{\theta}_{t+1} \doteq \boldsymbol{\theta}_t + \alpha \left(G_{t:t+1} - \hat{v}(S_t, \mathbf{w}) \right) \frac{\nabla \pi(A_t | S_t, \boldsymbol{\theta}_t)}{\pi(A_t | S_t, \boldsymbol{\theta}_t)}$$

- Even with the baseline, REINFORCE tends to show a higher variance due to the episode sampling
- How can you modify this based on the idea of TD approaches?

- Replace the target value!

$$\begin{aligned}\boldsymbol{\theta}_{t+1} &\doteq \boldsymbol{\theta}_t + \alpha \left(G_{t:t+1} - \hat{v}(S_t, \mathbf{w}) \right) \frac{\nabla \pi(A_t | S_t, \boldsymbol{\theta}_t)}{\pi(A_t | S_t, \boldsymbol{\theta}_t)} \\ &= \boldsymbol{\theta}_t + \alpha \left(R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w}) - \hat{v}(S_t, \mathbf{w}) \right) \frac{\nabla \pi(A_t | S_t, \boldsymbol{\theta}_t)}{\pi(A_t | S_t, \boldsymbol{\theta}_t)} \\ &= \boldsymbol{\theta}_t + \alpha \delta_t \frac{\nabla \pi(A_t | S_t, \boldsymbol{\theta}_t)}{\pi(A_t | S_t, \boldsymbol{\theta}_t)}.\end{aligned}$$

One-step Actor–Critic (episodic), for estimating $\pi_{\theta} \approx \pi_*$

Input: a differentiable policy parameterization $\pi(a|s, \theta)$

Input: a differentiable state-value function parameterization $\hat{v}(s, \mathbf{w})$

Parameters: step sizes $\alpha^{\theta} > 0$, $\alpha^{\mathbf{w}} > 0$

Initialize policy parameter $\theta \in \mathbb{R}^{d'}$ and state-value weights $\mathbf{w} \in \mathbb{R}^d$ (e.g., to $\mathbf{0}$)

Loop forever (for each episode):

 Initialize S (first state of episode)

$I \leftarrow 1$

 Loop while S is not terminal (for each time step):

$A \sim \pi(\cdot|S, \theta)$

 Take action A , observe S', R

$\delta \leftarrow R + \gamma \hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w})$ (if S' is terminal, then $\hat{v}(S', \mathbf{w}) \doteq 0$)

$\mathbf{w} \leftarrow \mathbf{w} + \alpha^{\mathbf{w}} \delta \nabla \hat{v}(S, \mathbf{w})$

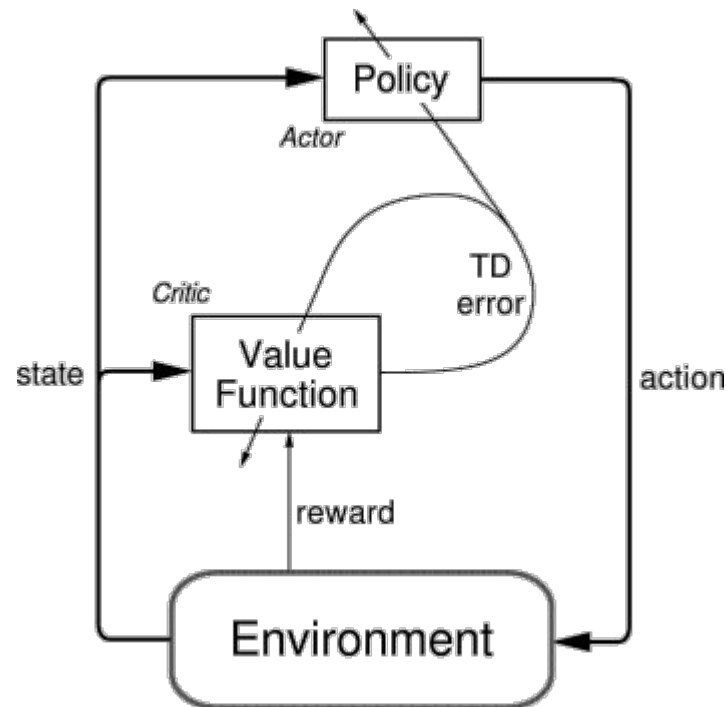
$\theta \leftarrow \theta + \alpha^{\theta} I \delta \nabla \ln \pi(A|S, \theta)$

$I \leftarrow \gamma I$

$S \leftarrow S'$

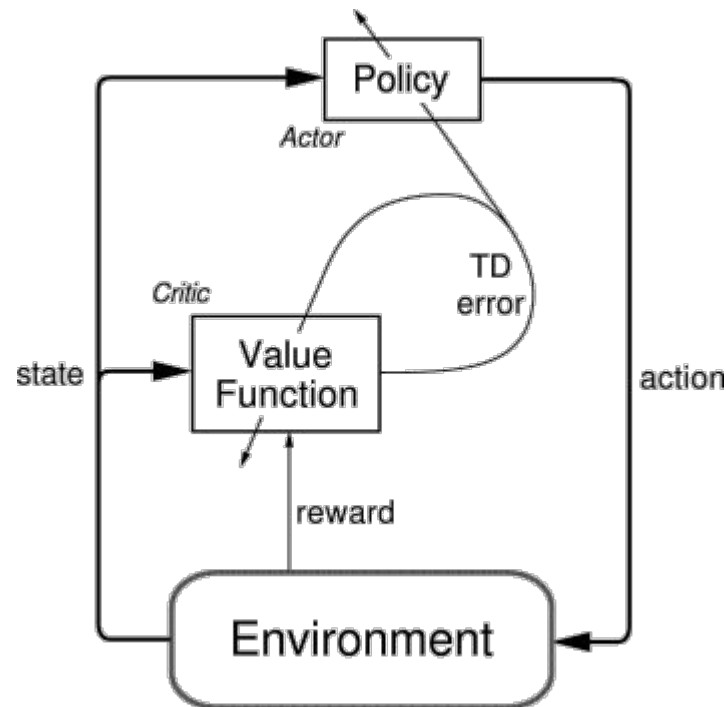
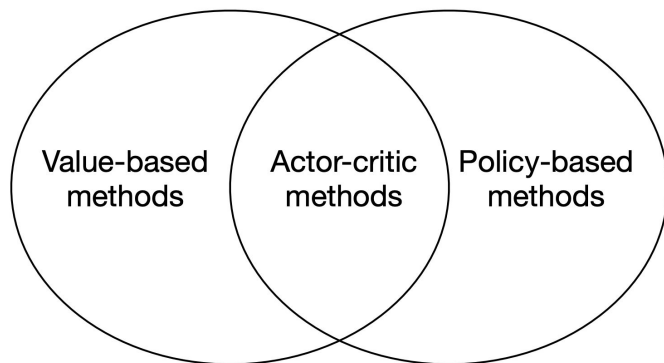
Actor-Critic Methods

- You have two separate entity:
 - **Actor**: Updates the policy function (parameters θ)
 - **Critic**: Updates the action-value function (parameters w)
- The action values estimated by the critic influence the policy update
- The bootstrapped one-step return is often superior to the actual return in terms of its variance



Actor-Critic Methods

- **Actor**: A policy-based method for the policy improvement
- **Critic**: A value-based method for the estimation of state values



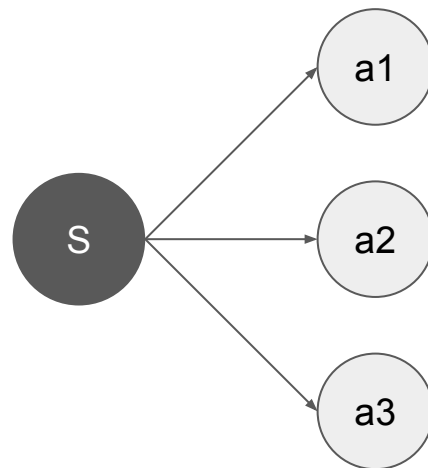
The Advantage Function

- What does it mean to use the TD error when updating the policy?

$$\delta_{\pi_{\theta}} = r + \gamma V_{\pi_{\theta}}(s') - V_{\pi_{\theta}}(s)$$

- This is similar to the baseline approach
- We compute the relative impact of taking action a based on the average performance at state s

$$\begin{aligned}\mathbb{E}_{\pi_{\theta}}[\delta_{\pi_{\theta}} \mid s, a] &= \mathbb{E}_{\pi_{\theta}}[r + \gamma V_{\pi_{\theta}}(s') \mid s, a] - V_{\pi_{\theta}}(s) \\ &= Q_{\pi_{\theta}}(s, a) - V_{\pi_{\theta}}(s) \\ &= A_{\pi_{\theta}}(s, a)\end{aligned}$$



Coding Exercise

